

IO614

Simulink Driver Block Manual

Contents

1	Description	3
1.1	IO614	3
2	Pin Mappings	4
2.1	IO614 Pin Mapping	4
3	Driver Blocks	8
3.1	IO614 - Setup	8
3.2	IO614 - CAN Write	11
3.3	IO614 - CAN Read	13
3.4	IO614 - CAN Status	15
3.5	IO614 - LIN Master Write	18
3.6	IO614 - LIN Master Read	19
3.7	IO614 - LIN Slave Write	22
3.8	IO614 - LIN Slave Read	24
3.9	IO614 - LIN Status	27
4	Usage Notes	29
4.1	IO614 Usage Notes	29

1 Description

1.1 IO614

Introduction

The IO614 offers four CAN HS ports.

Port 1 can be configured as CAN LS instead.

Port 2 contains an additional LIN port.

The ports are located on four DB9 - connectors. For a description of the pinout, refer to the I/O pin mapping section.

Driver Blocks

- Setup
- CAN Write
- CAN Read
- CAN Status
- LIN Master Write
- LIN Master Read
- LIN Slave Write
- LIN Slave Read
- LIN Status

Pin Mapping

- Pin mapping for IO614

Usage Notes

- Usage Notes for IO614

2 Pin Mappings

2.1 IO614 Pin Mapping

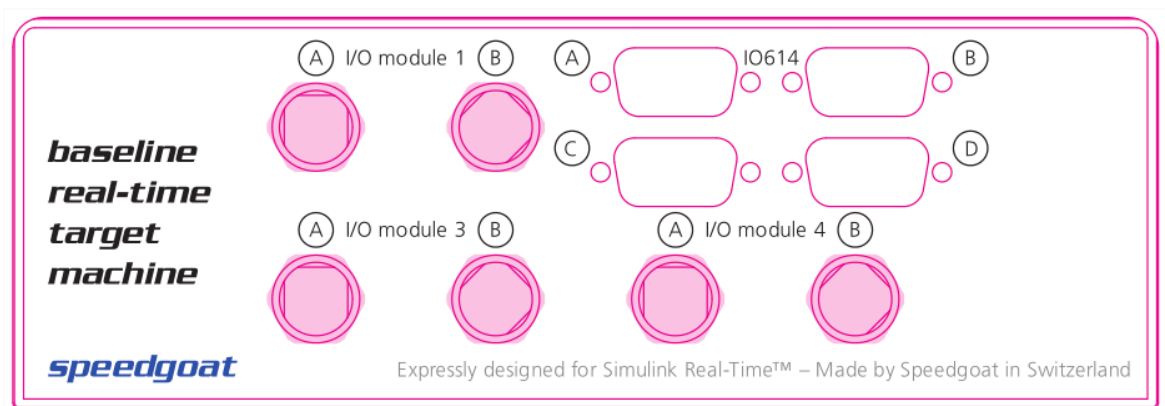
Description

The IO614 I/O module offers four DB9 connectors. Each connector contains a CAN port.

Additionally, there is one LIN port on the CAN port 2 connector.

Port Location

The following figure shows the outlines of an example Baseline target machine with the IO614 I/O module located in the top-right corner.



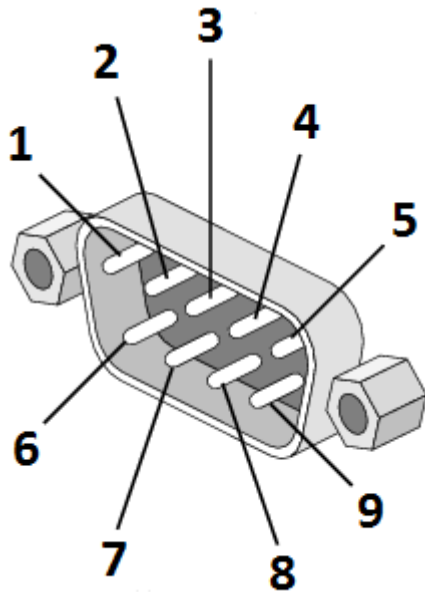
Note

Note that the four DB9 connectors are labeled A, B, C & D.

- Connector B contains CAN port 1
- Connector A contains CAN port 2
- Connector D contains CAN port 3
- Connector C contains CAN port 4

Connector Pin Mapping

The following figure shows the pin numbering of a DB9 connector when looking from the front (outside of the target machine).



The pin mapping of the two connectors is identical and is listed in the following table:

Pin	Signal	Condition
1	CAN Low	CAN port configured Low Speed (LS) (only port 1)
2	CAN Low	CAN port configured High Speed (HS)
3	GND	
4	Can High	CAN port configured Low Speed (LS) (only port 1)
5	-	
6	-	
7	CAN High	CAN port configured High Speed (HS)
8	LIN	Only port 2
9	LIN VBAT (8-18 VDC) ^a	Only port 2

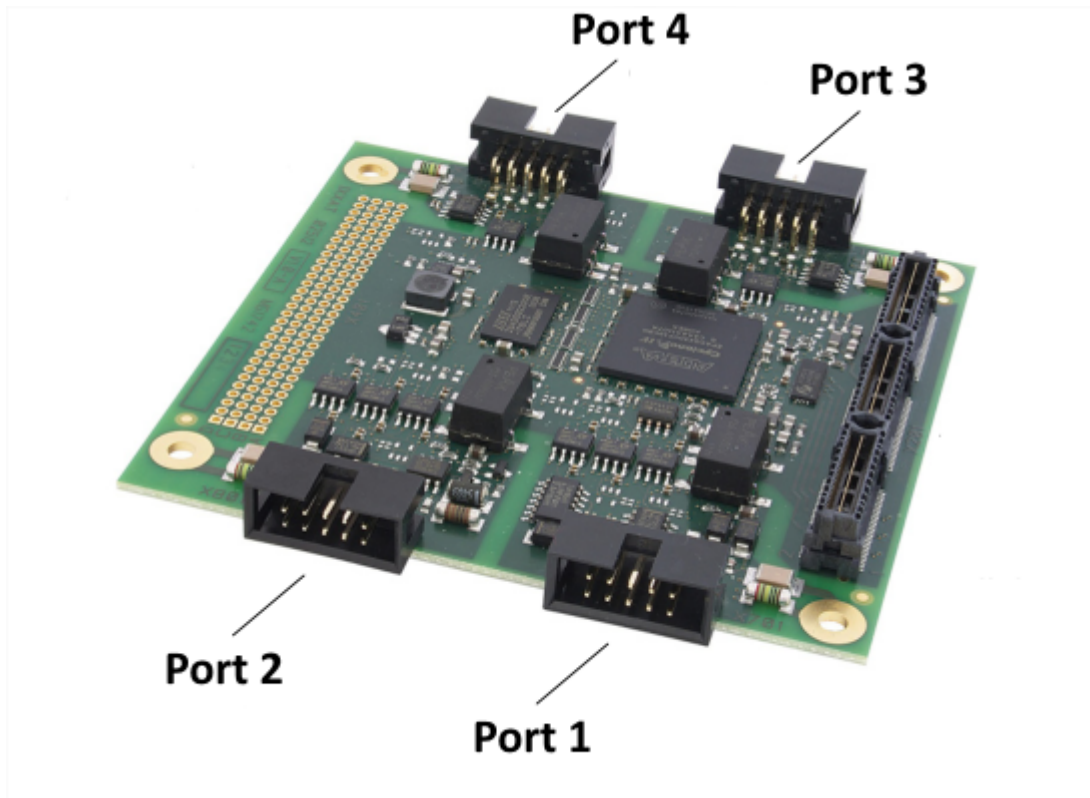
^aEvery LIN transceiver that belongs to the same LIN network needs to be supplied by the same voltage source.

For the CAN operation bus termination, a 120 Ω resistor is required between wires 2 and 7. The IO614 I/O module loopback cable provided by Speedgoat includes this resistor by default.

Raw Module

In rare cases, when the IO614 I/O module is used on a different target machine than the Baseline, the ports can be placed individually using boxed-header-to-DB9 adapter cables.

The ports on the module are ordered as shown in the following picture:

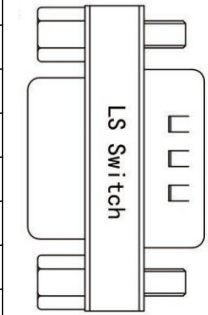


The pin mapping of the boxed headers is listed in the following table:

Pin	Signal	Condition
1	CAN Low	CAN port configured Low Speed (LS) (only port 1)
2	-	
3	CAN Low	CAN port configured High Speed (HS)
4	CAN High	CAN port configured High Speed (HS)
5	GND	
6	LIN	Only port 2
7	Can High	CAN port configured Low Speed (LS) (only port 1)
8	LIN VBAT (8-18VDC)	Only port 2
9	-	
10	-	

CAN LS Switch

The adapter provided routes the CAN LS signals to the same pins as CAN HS/FD. This means that the same cables can be used for CAN HS/FD and CAN LS operation. The CAN LS switch can be provided on request.

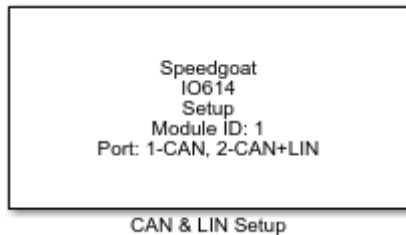
Pin Female Connector		Pin Male Connector
1		2
2		-
3		3
4		7
5		-
6		-
7		-
8		8
9		9

3 Driver Blocks

3.1 IO614 - Setup

Library

Simulink Real-Time - Speedgoat



Description

For each IO614 module in your target machine there should be exactly one "CAN & LIN Setup" driver block in your model. This block allows all the supported I/O types of this module to be configured, and therefore impacts on all the other driver blocks.

Ports

This driver block has no input or output ports.

Parameters

Module setup

Module ID

The Module ID must be unique among all IO611 Setup blocks in your model. It has two functions:

- It defines the logical connection to associate the Setup, Read and Write blocks with each other.
- It is relevant for the PCI slot auto-search feature: If only one IO614 is installed, the Module ID must be set to 1. If n modules are installed, it must be in the range 1:n. Not all the modules installed in the target machine need be used.

Drive letter on target machine

Choose on which drive this driver block should look for the firmware to load on the module, once the model is downloaded to the machine. The default setting for this parameter is 'C:', as the firmware will be located on the C drive by default. This only changes if a different drive letter is specified when using the `sg_download_FW()` function.

PCI slot (-1:autosearch)

There are two approaches for mapping the blocks to a specific IO614 module installed in the target machine. All IO614 modules must be configured using the same method.

- *Auto-Search:* The default value '-1' lets the real-time target machine auto-search for the I/O module. The priority is defined by the Module ID. The module with the lowest PCI bus/slot number is associated with Module ID 1, the next module found is associated to Module ID 2, and so on. Please refer to your target machine manual for the specific slot sequence.
- *Explicit Addressing:* The location of the module inside the target machine can be explicitly defined using the format '[BusNumber, SlotNumber]'. To determine these numbers, type sgio in the MATLAB command window.

Module Port Selection

In this group box, you can specify the configuration for each port. Note that LIN is only available on port 2.

CAN

This tab is only visible if at least one CAN port is enabled. There will be one group box per enabled CAN port, labeled with the number of the port. There is also one additional group box called Init./Term. Structures, where you can define CAN messages sent during initialization and termination of the CAN write driver blocks.

Operation Mode:

Choose High-Speed or Low-Speed operation. Low-Speed operation is according to ISO 11898-3. High-Speed operation is according to ISO 11898-2. Note that the Operation Mode is only configurable for port 1, as the other CAN ports only support High-Speed operation.

Baud rate

The baud rate for this CAN port. This control allows the most commonly used values to be selected. If special bit timing is needed, then select the User defined here and use the "User defined baud rate" parameter below.

User defined baud rate

If the defined User has been selected in the "Baud rate" parameter above, this parameter is used to configure the bit timing. The parameter is a vector with the following elements:

- Prescaler (BRP)
- Synchronization-Jump-Width (SJW)
- Time-Segment-1 (TSEG1)
- Time-Segment-2 (TSEG2)

For an explanation and calculation of the bit-timing values, check the free individual bit-timing table on can-wiki.info [<http://www.bittiming.can-wiki.info/>] using the following parameters:

- Table for: IFI CAN FD arbitration, in ISO mode
- Clock Rate: 80MHz
- Sample point: 87.5
- SJW: 1

Note that the Bitrate indicated on this website is in thousands (e.g. Bitrate = 1000 accords to 1Mbaud).

Standard Acceptance Mask

The rule to decide whether a CAN message with a standard ID (11 bits) is accepted on CAN port 1. The message is accepted if the following rule is true:

- $((ID \& StdMask) == StdCode)$

Set the StdMask element to zero (the default) to disable the rule. Note that this parameter takes a decimal value. See the "Usage notes" section for examples of how to set this parameter.

Extended Acceptance Mask

The rule to decide whether a CAN message with an extended ID (29 bits) is accepted on CAN port 1. The message is accepted if either of the following rules is true:

- $((ID \& ExtMask) == ExtCode)$

Set the ExtMask element to zero (the default) to disable the rule. Note that this parameter takes a decimal value. See the "Usage notes" section for examples of how to set this parameter.

Initialization Structures

Defines CAN messages sent during initialization of the Setup block. For more information, see: Initialization and Termination CAN Messages.

Termination Structures

Defines CAN messages sent during termination of the Setup block. For more information, see: Initialization and Termination CAN Messages.

LIN

This tab is only visible if LIN on port 2 is enabled.

Operation Mode

The IO614 can be configured to act as a master or a slave. In the LIN topology, only one master is allowed per network.

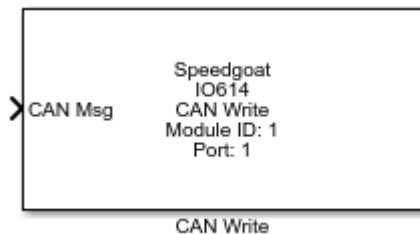
Baud rate

This parameter is valid for master and slave operation. Range between 1000 and 20000.

3.2 IO614 - CAN Write

Library

Simulink Real-Time - Speedgoat



Description

Each execution of a "CAN Write" block represents a CAN message being sent to the IO61X output buffer. Multiple instances of the block can be used to send messages at multiple rates.

The write buffer has the following size:

Module	Write Buffer Size
IO611	96 messages
IO612	101 messages
IO613	96 messages
IO614	101 messages

Depending on the selected baud rate and the "CAN Write" block sample times, it is important to not let the output buffer overflow. You can use the block status output port to verify this.

Ports

Which inputs are available depends on the "Message type" mask setting:

- CAN: Only CAN message output is available
- RAW: Data, Identifier, Length, Identifier Type and Timestamp outputs are available

Inputs

CAN Msg

The complete CAN message. Please use the "CAN Pack" block to create this message.

Data Types: `CAN_MESSAGE`

ID

The CAN message identifier.

Data Types: `uint32`

EFF

The identifier type (Extended Frame Format) of the CAN message.

- 0: 11-bit identifier
- 1: 29-bit extended identifier

Data Types: `uint32`

DLC

The length of the payload data.

Data Types: `uint32`

Data

The payload data of the CAN message is 8 bytes.

Data Types: `uint8(8)`

Outputs**Status**

Indicates if the message supplied to the input ports has been successfully placed in the IO601 I/O module tx queue.

- 0: The message was not successfully written to the send buffer.
- 1: The message was successfully written to the send buffer.

This port can be enabled or disabled from the "Show status output port" in the dialog box for this driver block.

Data Types: `uint32`

Parameters**Module ID**

Specifies the ID of the IO61X module this driver block should be connected to. It must correspond to the module ID of the according setup block. Check the setup block help for more details.

Sample Time

Defines the base sample time at which this driver block gets its sample hit. It can also be set to -1 for inherited sample time. The units are in seconds.

Send Message with Port

Choose the CAN port to which this driver block should be connected.

Message Type

- CAN: Use "CAN Pack" block to define the CAN message.
- RAW: Manual definition of the CAN message without the "CAN Pack" block. Please see the CAN specification for the correct input values for the Identifier, Identifier Type, Length and Data.

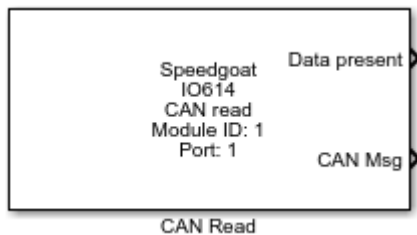
Enable Status Output Port

Adds the "Status" output port.

3.3 IO614 - CAN Read

Library

Simulink Real-Time - Speedgoat



Description

The "CAN Read" block reads messages from one port of the IO61X input queue.

In each execution step it reads one message and indicates if more messages are available.

See the "Usage Notes" to find out how this block can be used in a "do...while" subsystem to prevent a possible overflow of the read buffer.

The read buffer has a capacity of 501 messages per port.

Ports

Which output ports are available depends on the "Message type" mask setting:

- CAN: Only CAN message output is available.
- RAW: Data, Identifier, Length, Identifier Type and Timestamp outputs are available.
- RAW & CAN: All output ports are available.

Outputs

Data Present

Status signal is "1" if new data is available and "0" otherwise.

Data Types: `boolean`

CAN Msg

The complete CAN message. Please use the "CAN Unpack" block to resolve the data.

Data Types: `CAN_MESSAGE`

ID

The CAN message identifier.

Data Types: `uint32`

EFF

The identifier type (Extended Frame Format) of the CAN message.

- 0: 11-bit identifier
- 1: 29-bit extended identifier

Data Types: `uint32`

DLC

Length of the payload data.

Data Types: `uint32`

Timestamp

The timestamp for receipt of the CAN message. The resolution is 1 μ s, so in order to get a double value in seconds, the block output must be converted to double and multiplied by 1e-6. This 1 μ s resolution is given by the hardware implementation of the IO61X module. The time is reset to zero at each model start.

Data Types: `uint32`

Data

The payload data of the CAN message.

Data Types: `uint8(8)`

Parameters**Module ID**

Specifies the ID of the IO61X module this driver block should be connected to. It must correspond to the module ID of the according setup block. Check the setup block help for more details.

Sample Time

Defines the base sample time at which this driver block gets its sample hit. It can also be set to -1 for inherited sample time. The units are in seconds.

Message Type

Defines how the CAN message is represented at the block outputs.

- CAN: Only one "CAN Msg" output port is available. It will always be connected to one or multiple "CAN Unpack" blocks.
- RAW: The message content is presented through five output ports: "Data", "ID", "DLC", "FF" and "Timestamp".
- CAN + RAW: Both versions are displayed.

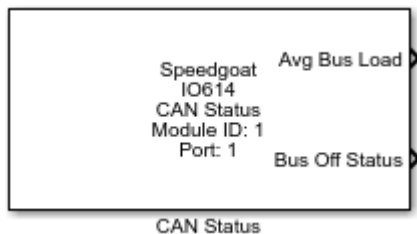
Read Message from Port

Choose the CAN port to which this driver block should be connected.

3.4 IO614 - CAN Status

Library

Simulink Real-Time - Speedgoat



Description

This block provides CAN bus and controller status information.

Ports

Outputs

All

Status information of the CAN port and the controller. The individual signals can be enabled or disabled in the driver mask. See the "Parameters" section.

Inputs

Bus Reco

With "1" impulse, the Status block will attempt to recover from a BUS OFF state. After bus restart, the value should be set to zero.

Data Types: uint32

Parameters

Setup

Module ID

Specifies the ID of the IO61X module to which this driver block should be connected. It must correspond to the module ID of the according setup block. Check the setup block help for more details.

Sample Time

Defines the base sample time at which this driver block gets its sample hit. It can also be set to -1 for inherited sample time. The units are in seconds.

Get Status for Port

Choose the CAN-FD port to which this driver block should give status information.

Status Selection

Choose which types of status information to access with this driver block. The selectable status outputs for this block in the Status-Tab will change accordingly.

- Port: Status information about the CAN-FD port to which this driver block is mapped.
- Controller: Configuration information about the CAN controller of the port to which this driver block is mapped.
- All: Both of the above.

Bus OFF

Defines if and when the CAN controller will try to recover from a "BUS OFF" status.

- Off: The bus will stay OFF, no recovery is attempted.
- Auto: The Status block will restart the CAN stack for both ports to recover from the BUS OFF state.
- Manual: The "Bus Reco" Input is added. Set to "1" for a Manual bus restart.

Status

Here, the user can select the required status signals. The selected signals appear as block outputs.

Average Bus Load (0 – 100%)

Average bus load in percent. The CAN controller computes the bus load every 4096 bits. In most cases it is sufficient to request the bus load once per second. For example, for 500 kBaud: $500\text{kBaud} = 500\text{bit/ms} \Rightarrow 2\mu\text{s/bit} * 4096\text{bit} = 8.2\text{ms}$ update time for the bus load.

Operating Mode

- 0: Current operating mode of the CAN controller is Lowspeed (LS).
- 1: Current operating mode of the CAN controller is Highspeed (HS).

Prescaler (BPS)

This value corresponds to the BPS value of the selected baud rate in the corresponding setup block.

Time Segment 1 (TSEG1)

This value corresponds to the TSEG1 value of the selected baud rate in the corresponding setup block.

Time Segment 2 (TSEG2)

This value corresponds to the TSEG2 value of the selected baud rate in the corresponding setup block.

Synchronization Jump Width (SJW)

This value corresponds to the SJW value of the selected baud rate in the corresponding setup block.

Transmission Pending

The CAN controller is sending a message.

Data Overrun Occurred

This flag will be set if the receive queue is full and the CAN controller cannot write the next received message to the queue. To reset the flag, the controller must be stopped and started again.

Error Warning Limit Exceeded

Each CAN controller has an internal error counter which counts physical bus problems.

- 0: Error counter value is lower than the error warning level (<96).
- 1: Error counter has reached the error warning level limit (≥ 96).

Bus Off Status

Uses the same internal error counter as the warning output above:

- 0: Error counter value is lower than the maximum error level (< 255).
- 1: Error counter has reached the maximum error level limit ($= 255$). Bus will be shut down.

Init Mode Active

- 0: The CAN controller is in the initialization phase and currently is not started.
- 1: The CAN controller is started.

Bus Coupling Error

Indicates a bus coupling error if CAN LS is used.

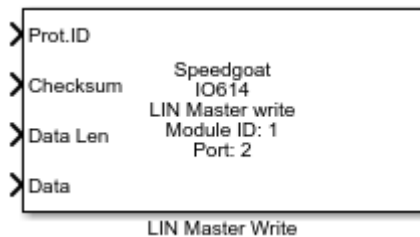
Bus Recovery Counter

This uint32 counter is incremented each time the bus-recovery process of the CAN port is finished.

3.5 IO614 - LIN Master Write

Library

Simulink Real-Time - Speedgoat



Description

Send messages over LIN.

Ports

All the input ports of this block are vectors with as many elements as messages are configured to be sent per sample hit. Except the number of elements of the data input port is 8 times the number of configured messages.

Inputs

Prot.ID

The LIN identifier (PID). Range 0 to 63.

Data Types: `uint8`

Checksum

Checksum version to use:

- 0: Classic Checksum (LIN 1.1, LIN 1.2, LIN 1.3)
- 1: Enhanced Checksum (LIN 2.0, LIN 2.1, LIN 2.2)

Data Types: `uint8`

Data Len

Data length code (DLC) for the message to send. Range 0 to 8

Data Types: `uint8`

Data

The message data. The LIN protocol allows for messages with up to eight data bytes. If N messages are configured to be sent per sample hit, this vector will simply be sized $N * 8$.

Data Types: `uint8`

Parameters

Module ID

Specifies the ID of the IO61X module this driver block should be connected to. It must correspond to the module ID of the according setup block. Check the setup block help for more details.

Sample Time

Defines the base sample time at which this driver block gets its sample hit. It can also be set to -1 for inherited sample time. The units are in seconds.

Send Message with Port

Choose the LIN port to which this driver block should be connected.

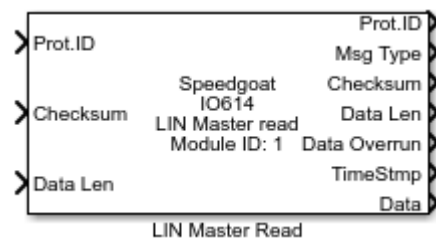
Number of LIN Messages

This field configures how many LIN messages to send per sample hit. This parameter directly sets the input port width. Range is 1 to 50.

3.6 IO614 - LIN Master Read

Library

Simulink Real-Time - Speedgoat



Description

Read messages from the LIN port

Ports

All the input and output ports of this block are vectors with as many elements as messages are configured to be read per sample hit. Except the number of elements of the data output port is 8 times the number of configured messages.

Per configured message, there will be a read transfer on the LIN bus every sample hit. The received message is put to the outputs within the same sample time.

Inputs

Prot.ID

The LIN identifier (PID). Range 0 to 63.

Data Types: uint8

Checksum

Checksum version to use in the read request:

- 0: Classic Checksum (LIN 1.1, LIN 1.2, LIN 1.3)
- 1: Enhanced Checksum (LIN 2.0, LIN 2.1, LIN 2.2)

Data Types: `uint8`

Data Len

Data length code (DLC) for the read request. Range 0 to 8.

Data Types: `uint8`

Outputs**Prot.ID**

The LIN identifier received. Range 0 to 63. Parity P0 and P1 are removed by the driver.

If an error message has been received (See output "Msg Type" below), this output will contain the error code. The following error codes are possible:

- 0: no error
- 1: bit error
- 2: checksum error
- 3: parity error
- 4: no answer from slave
- 5: Sync break error
- 6: Inconsistent sync break field
- 7: More data expected
- 8: Timeout after start of sync break field
- 9: Timeout after sync break field
- 10: Timeout after sync field
- 11: Slave didn't start
- 12: LIN parameter error

Data Types: `uint32`

Msg Type

This port can take three possible values:

- 0: Dataframe message
- 2: Error message
- 3: Wakeup message

If the message type is "Error message", the error code is written to the "Prot.ID" output. The error codes are listed above.

Data Types: uint32

Checksum

Checksum version used in the received message:

- 0: Classic Checksum (LIN 1.1, LIN 1.2, LIN 1.3)
- 1: Enhanced Checksum (LIN 2.0, LIN 2.1, LIN 2.2)

Data Types: uint32

Data Len

Data length code (DLC) of the received message. Range 0 to 8.

Data Types: uint32

Data Overrun

This field contains 3 bits of message information.

- Bit 0: 1 signalizes an overrun of the data buffer
- Bit 1: 0 if message received, 1 if message sent (self-receipt)
- Bit 2: 1 if the message contains no data

In case of a self-receipt, this output will be equal to 2 (bit 1 set).

Data Types: uint32

Timestamp

The timestamp for receipt of the LIN message. The resolution is 1µs, so in order to get a double value in seconds, the block output must be converted to double and multiplied by 1e-6. This 1µs resolution is given by the hardware implementation of the IO61X modules. The time is reset to zero at each model start.

Data Types: uint32

Data

The message data. The LIN protocol allows for messages with up to eight data bytes. If N messages are configured to be received per sample hit, this vector will simply be sized N * 8.

Data Types: uint8

Parameters

Module ID

Specifies the ID of the IO61X module this driver block should be connected to. It must correspond to the module ID of the according setup block. Check the setup block help for more details.

Sample Time

Defines the base sample time at which this driver block gets its sample hit. It can also be set to -1 for inherited sample time. The units are in seconds.

Read Message from Port

Choose the LIN port to which this driver block should be connected.

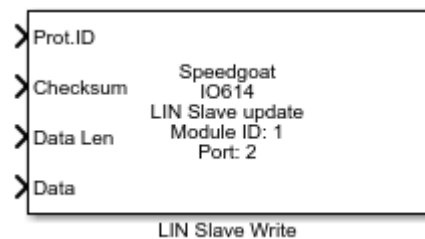
Number of LIN Messages

Configures how many read requests to perform per sample hit. Input and output port widths depend directly on this field. Range is 1 to 50.

3.7 IO614 - LIN Slave Write

Library

Simulink Real-Time - Speedgoat



Description

Write to the Transmit Buffers of the LIN Slave.

Ports

All the input ports of this block are vectors with as many elements as values are configured to be updated per sample hit. Except the number of elements of the data input port is 8 times the number of configured messages.

Inputs

Prot.ID

The LIN identifier (PID). Range 0 to 63.

Data Types: `uint8`

Checksum

Checksum version to use:

- 0: Classic Checksum (LIN 1.1, LIN 1.2, LIN 1.3)
- 1: Enhanced Checksum (LIN 2.0, LIN 2.1, LIN 2.2)

Data Types: `uint8`

Data Len

Data length code (DLC). Range 0 to 8.

Data Types: `uint8`

Data

The new data. The LIN protocol allows for messages with up to eight data bytes. If N values in the buffer are configured to be updated per sample hit, this vector will simply be sized $N * 8$.

Data Types: `uint8`

Parameters**Module ID**

Specifies the ID of the IO61X module this driver block should be connected to. It must correspond to the module ID of the according setup block. Check the setup block help for more details.

Sample Time

Defines the base sample time at which this driver block gets its sample hit. It can also be set to -1 for inherited sample time. The units are in seconds.

Send Message with Port

Choose the LIN port to which this driver block should be connected.

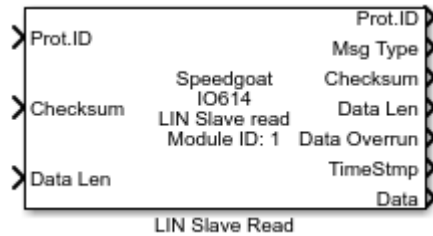
Number of LIN Messages

Configures how many values in the buffer to update per sample hit. This parameter directly sets the input port width. Range is between 1 and 50.

3.8 IO614 - LIN Slave Read

Library

Simulink Real-Time - Speedgoat



Description

Read messages from the LIN port.

Ports

All the input and output ports of this block are vectors with as many elements as messages are configured to be read. Except the number of elements of the data output port is 8 times the number of configured messages.

The input ports of this driver block are used to filter incoming messages. Note that the Checksum can be changed at runtime, whereas the Message IDs cannot.

The outputs will always show the last received message that complies with the relevant filter configured with the inputs.

Inputs

Prot.ID

Message ID for the message filter

Data Types: uint8

Checksum

Checksum for the message filter

Data Types: uint8

Data Len

Data length for the message filter

Data Types: uint8

Outputs

Prot.ID

The LIN identifier received. Range 0 to 63. Parity P0 and P1 are removed by the driver.

If an Error message has been received (See output "Msg Type" below), this output will contain the error code. The following error codes are possible:

- 0: no error
- 1: bit error
- 2: checksum error
- 3: parity error
- 4: no answer from slave
- 5: Sync break error
- 6: Inconsistent sync break field
- 7: More data expected
- 8: Timeout after start of sync break field
- 9: Timeout after sync break field
- 10: Timeout after sync field
- 11: Slave didn't start
- 12: LIN parameter error

Data Types: `uint32`

Msg Type

This port can take three possible values:

- 0: Dataframe message
- 2: Error message
- 3: Wakeup message

If the message type is "Error message", the error code is written to the "Prot.ID" output. The error codes are listed above.

Data Types: `uint32`

Checksum

Checksum version used in the received message:

- 0: Classic Checksum (LIN 1.1, LIN 1.2, LIN 1.3)
- 1: Enhanced Checksum (LIN 2.0, LIN 2.1, LIN 2.2)

Data Types: `uint32`

Data Len

Data length code (DLC) of the received message. Range 0 to 8

Data Types: `uint32`

Data Overrun

State of the received message queue.

- 0 if normal operation
- 1 if the buffer is overrun

Data Types: `uint32`

Timestamp

The timestamp for receipt of the LIN message. The resolution is 1 μ s, so in order to get a double value in seconds, the block output must be converted to double and multiplied by 1e-6. This 1 μ s resolution is given by the hardware implementation of the IO61X modules. The time is reset to zero at each model start.

Data Types: `uint32`

Data

The message data. The LIN protocol allows for messages with up to eight data bytes. If N messages filtered for, this vector will simply be sized N * 8.

Data Types: `uint8`

Parameters**Module ID**

Specifies the ID of the IO61X module this driver block should be connected to. It must correspond to the module ID of the according setup block. Check the setup block help for more details.

Sample Time

Defines the base sample time at which this driver block gets its sample hit. It can also be set to -1 for inherited sample time. The units are in seconds.

Read Message from Port

Choose the LIN port to which this driver block should be connected.

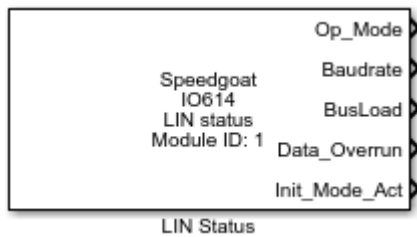
Number of LIN Messages

This field configures how many LIN messages to filter for. This parameter directly sets the input and output port widths. Range is between 1 and 50.

3.9 IO614 - LIN Status

Library

Simulink Real-Time - Speedgoat



Description

This block provides status information from the LIN port.

Ports

Outputs

Op_Mode

This port can have the following values:

- 0: the board acts as a slave
- 3: the board acts as a master

Data Types: `uint32`

Baud Rate

The current configured baud rate.

Data Types: `uint32`

BusLoad

The current percentage (0..100) of LIN bus load.

Data Types: `uint32`

Data_Overrun

1 if data overrun occurred.

Data Types: `uint32`

Init_Mode_Act

1 if LIN controller is still in init mode.

Data Types: `uint32`

Parameters

Block Setup

Module ID

Specifies the ID of the IO61X module this driver block should be connected to. It must correspond to the module ID of the according setup block. Check the setup block help for more details.

Sample Time

Defines the base sample time at which this driver block gets its sample hit. It can also be set to -1 for inherited sample time. The units are in seconds.

Get Status for Port

Choose the LIN port to which this driver block should be connected.

Status

In the "Status" tab, the user can select the required status signals. The selected signals appear as block outputs.

4 Usage Notes

4.1 IO614 Usage Notes

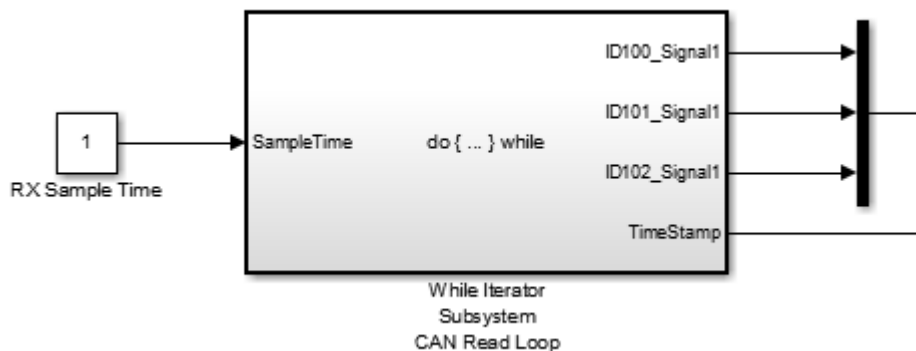
CAN Read Strategy

The "CAN Read" block reads one message from one port of the module input queue. The "Data present" output port shows if there are more messages available in the queue.

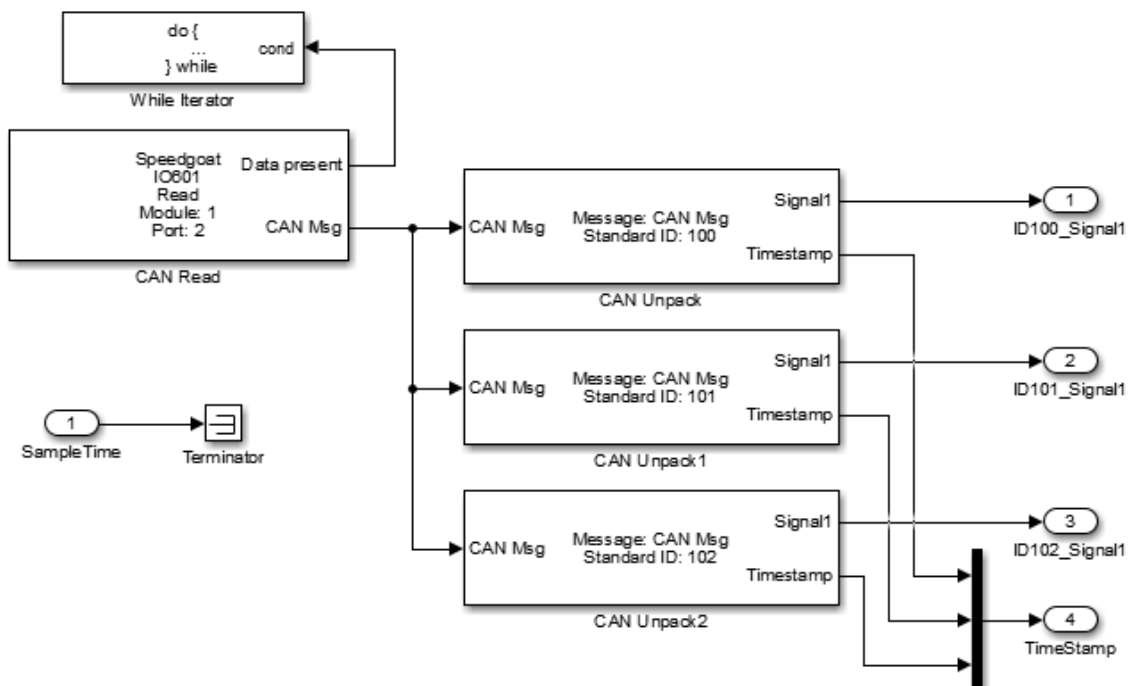
If your model runs faster than the CAN messages come in, it is safe to read just one message on each model execution step. If, however, there is a chance that more than one CAN message might be received per step, you might want to read all the available messages.

This is most conveniently done with a "While Iterator" subsystem.

Top Level View:



Inside the While Iterator Subsystem



The "Sample Time" input port defines the rate at which the subsystem is executed. Otherwise it will run on the base rate.

CAN Message Filtering

The module has some limited message identifier filtering capabilities. The filters are handled on the module level, so rejected messages do not add to the model execution latency.

Here are two examples of how to configure CAN message ID filtering. Both of these work with the Acceptance parameter for extended CAN IDs ('Acceptance [ExtMask1, ExtCode1, ExtMask2, ExtCode2]:'). The Acceptance parameter for standard IDs works in the same way.

These parameters are configured in the Setup driver block.

To filter specific IDs:

ExtMask1 = double(hex2dec('1FFFFFFF')) - masks all 29 bits of the ID

ExtCode1 = double(hex2dec('7D0')) - first ID to filter is 0x7D0

ExtMask2 = double(hex2dec('1FFFFFFF')) - masks all 29 bits of the ID

ExtCode2 = double(hex2dec('FFFF')) - second ID to filter is 0xFFFF

In this case, only the message IDs 0x7D0 and 0xFFFF are accepted.

To filter a group of IDs smaller than a certain value:

ExtMask1 = double(hex2dec('1FFFFF00'))

ExtCode1 = double(hex2dec('00000000'))

ExtMask2 = double(hex2dec('1FFFFF00'))

ExtCode2 = double(hex2dec('00000000'))

In this case, only IDs below 0xFF (255) are accepted.

Note

The Acceptance parameter takes decimal values. As it is often easier to work with hex for CAN IDs, we use MATLAB conversion functions here for simplicity.

Initialization and Termination CAN Messages

The CAN Setup driver blocks for the supported CAN boards allow you to define CAN messages to be sent during initialization and termination of the real-time application. The driver blocks send these messages once at the beginning of each application run and once before an application run is stopped. The main purpose for sending these messages is to initialize or terminate other CAN nodes on the network, such as CANopen or DeviceNet nodes. Even if those CAN layers are not directly supported, the model can usually communicate with those nodes with standard CAN messages, provided the nodes are initialized. The initialization and termination fields of the Setup blocks are intended for this purpose.

You define the initialization and termination CAN messages by using MATLAB® struct arrays with CAN specific field names. This approach was also used for the RS-232 and general counter driver blocks found in the Simulink® Real-Time™ I/O library. Refer to those driver blocks and their help for additional information about this basic concept.

The CAN Setup block-specific field names are as follows:

- Port: Selects the CAN port over which the message is sent. Valid values are either 1 or 2 (double).
- Protocol: Select the CAN protocol. Valid values are either "CAN" or "CAN-FD" (character vectors).
- BRS: Defines if fast data baudrate should be used for CAN-FD. Valid values are either 0 for standard baudrate or 1 for fast data baudrate (double).
- Type: Defines whether the message to be sent is of type standard or extended. Valid values are either 'Standard' or 'Extended' (character vectors).
- Identifier: Defines the identifier of the message. The value (scalar) itself must be in the corresponding identifier range (standard or extended).
- Data: Defines the data frame to be sent out along with the CAN message. The length of the row vector defines the data frame size.
- If the CAN message data type is a CAN_MESSAGE data type, the value of the frame size vector must match the Length parameter of the connected CAN Pack block.
- If the CAN message data type is a double, this field defines the data frame to be sent out along with the CAN message. The value must be a row vector of type double with a maximum length of 8. Each element of the vector defines 1 byte. In this case, the first element defines the data for byte 0 and the eighth element the data for byte 7. Each element can have a value from 0 through 255 (decimal).
- Pause: Defines the amount of time, in seconds, that the Setup block waits after this message has been sent before sending the next message. Valid values are 0–0.05 seconds. Some CAN nodes need time to settle before they can accept the next message. For example, the node must settle when the previous message puts it in a new operational mode. Use this field to specify the idle times (double).

Compiler

This I/O Module does not support Microsoft Windows SDK 7.1. Please install Microsoft Visual Studio 2010 or later.

Different block rates are only supported with Microsoft Visual Studio 2012 or later.